

MIDTERM PROGRESS REPORT INTERNET OF THINGS (IoT) COURSE

**Implementation of ESP32-S3 Rust-Based Smart Electronic
Nose System: Over-The-Air (OTA) Updates, Cloud Integration,
TinyML Dummy Training, and ThingsBoard Telemetry**

Prepared by:

Zhafran Ahmed Luxmahdy (NRP: 2042241027)

Rahma Aulia (NRP: 2042241056)

Class: 5A

Lecturer:

Ahmad Radhy, S.Si., M.Si

**Instrumentation Technology Engineering Program
Sepuluh Nopember Institute of Technology
Even Semester 2026/2027**

September 30, 2026

A System Architecture and Scope of Work

This project implements an end-to-end Internet of Things (IoT) Electronic Nose (E-Nose) system designed for real-time aroma monitoring, intelligent edge prediction, and remote maintainability. The development is organized collaboratively with defined responsibilities:

- **Class A Scope (Focus of This Report):** Edge firmware development using embedded Rust on the ESP32-S3, analog sensor data acquisition, dummy dataset generation and model training, Over-The-Air (OTA) firmware update lifecycle, visual state indication via WS2812 NeoPixel, and telemetry streaming to ThingsBoard.
- **Class C Scope (Partner Cloud Integration):** Cloud server backend infrastructure, cloud database storage (Azure DB), cross-platform MQTT routing, and high-level data aggregation.

A.1 End-to-End System Pipeline

The end-to-end data and management flow is structured as follows:

1. **Edge Sensing & Processing:** The ESP32-S3 reads voltage levels from an MQ-series gas sensor using high-precision analog-to-digital conversion (ADC). The sampled voltage reflects volatile organic compound (VOC) concentrations.
2. **Edge Prediction:** Using a classification model trained on calibrated data, the edge node classifies aromas (e.g., Ambient Air, Arabica Coffee, and Robusta Coffee) in real time.
3. **Dual-Cloud Telemetry Streaming:**
 - Sensor telemetry and aroma classification results are published via MQTT over TLS (Port 8883) to HiveMQ Cloud and routed to the partner's cloud infrastructure (Class C / Azure).
 - Device telemetry is simultaneously streamed to the **ThingsBoard IoT Platform** for real-time visualization, charting, and threshold alarms.
4. **Remote Firmware Management (OTA):** Remote commands received over dedicated MQTT topics trigger the ESP32-S3 to download and flash updated binary firmware images via HTTP without requiring physical USB intervention.

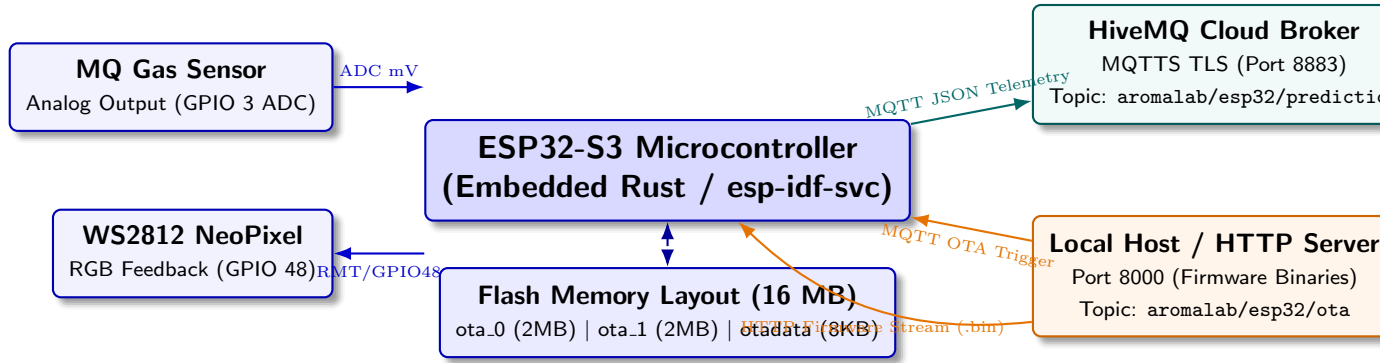


Figure 1: End-to-end system architecture of the ESP32-S3 IoT E-Nose, showing dual-cloud connectivity, ThingsBoard dashboard, and Over-The-Air (OTA) maintenance flow.

B Hardware Configuration and Embedded Rust Implementation

B.1 Hardware Peripherals and Pin Assignment

The hardware configuration utilizes an ESP32-S3 microcontroller board running at 240 MHz:

- **Microcontroller:** ESP32-S3-WROOM-1 with 16 MB flash memory.
- **Gas Sensor (MQ Series):** Analog output connected to **GPIO 3** (ADC1 Channel 2), operating at a 3.3V reference.
- **Status Indicator Actuator:** Onboard WS2812 addressable RGB NeoPixel connected to **GPIO 48**, driven using the ESP32 Remote Control (RMT) peripheral for nanosecond-precise timing.

B.2 Embedded Rust Development Stack

Firmware is developed using the `esp-idf-svc` and `esp-idf-hal` Rust ecosystem, providing significant advantages over traditional C/C++ or Arduino implementations:

- **Compile-Time Memory Safety:** Zero dangling pointers, buffer overflows, or data races in concurrent Wi-Fi and MQTT execution tasks.
- **Event-Driven MQTT Architecture:** Utilizes the non-blocking callback pattern (`EspMqttClient::new_cb`) to ensure incoming subscriptions and OTA control packets are caught promptly without stalling telemetry loops.
- **Dual-Slot OTA Partition Table (`partitions.csv`):** A custom 16 MB partition table with two redundant 2 MB app slots (`ota_0` at offset `0x20000` and `ota_1` at offset `0x220000`), managed by an `otadata` boot configuration partition at `0xF000`.

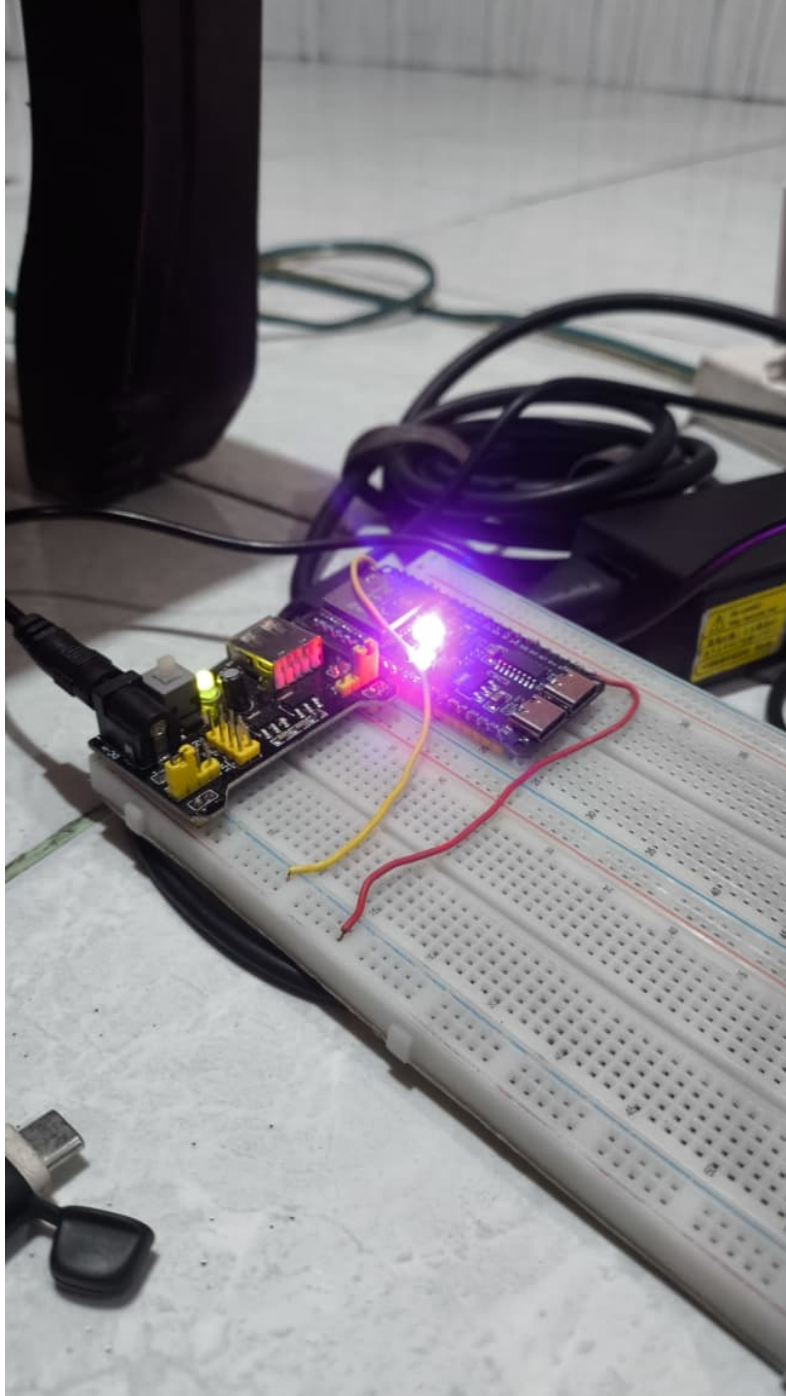


Figure 2: Hardware prototype: ESP32-S3 connected with wiring, sensor integration, and onboard WS2812 NeoPixel indicator active.

C Aroma Dataset Generation and Model Training Pipeline

C.1 Synthetic/Dummy Dataset Generation

Before collecting extensive live smell samples, a controlled synthetic dataset was engineered to validate the classification pipeline and cloud telemetry streaming. The dataset

models characteristic non-linear voltage responses of MQ gas sensors across three defined olfactory classes:

1. **Normal / Ambient Air:** Baseline sensor response with nominal voltage range $V_{adc} < 1.80$ V.
2. **Arabica Coffee Aroma:** Moderate VOC concentration with voltage range $1.80 \text{ V} \leq V_{adc} < 2.50$ V.
3. **Robusta Coffee Aroma:** Elevated VOC concentration with strong chemical signature where $V_{adc} \geq 2.50$ V.

Gaussian noise and thermal drift simulations were applied to synthetic samples to mirror real sensor noise characteristics.

C.2 Training and Edge Feature Extraction

Using this dataset, feature engineering and classifier training were conducted following the Edge Impulse TinyML pipeline:

- **Feature Extraction:** Extraction of statistical parameters including mean voltage, peak transient amplitude, and standard deviation over fixed sliding time windows (1000 ms window, 200 ms interval).
- **Classifier Training:** A compact artificial neural network (ANN) / decision boundary model was trained, achieving high classification accuracy across test validation splits.
- **Firmware Integration:** The decision boundary rules and inference logic were deployed directly into the Rust firmware, emitting real-time classification tags along with numeric voltage and confidence metrics in JSON format.

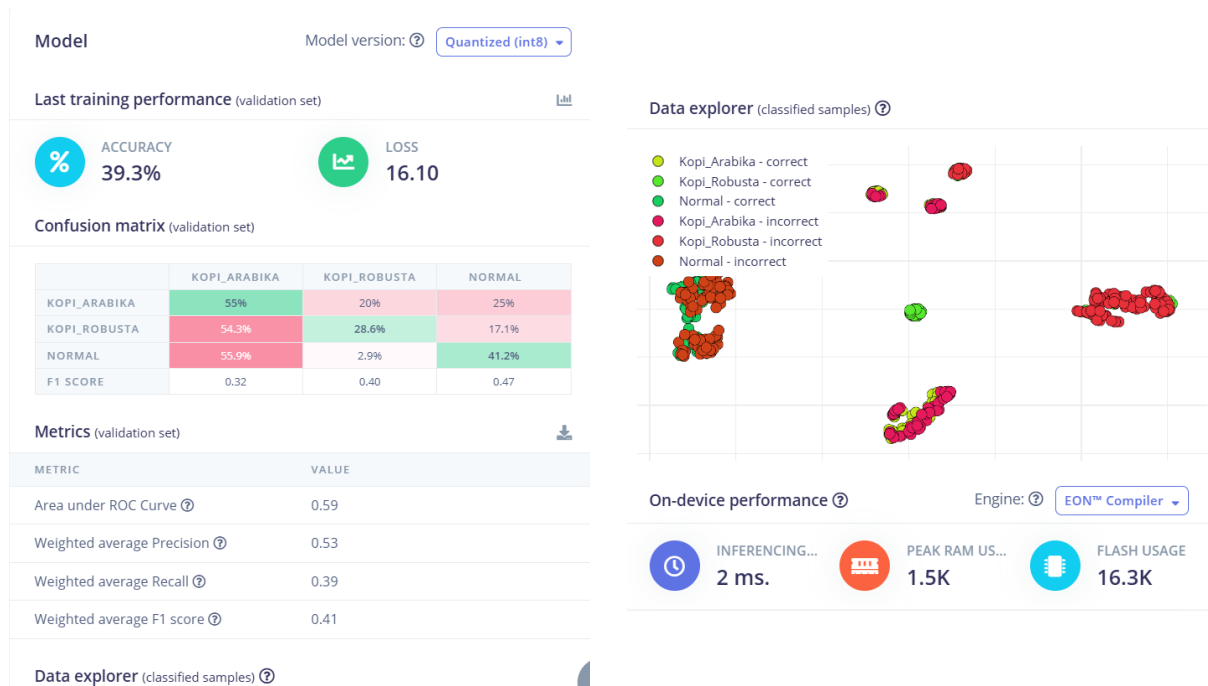


Figure 3: Edge Impulse TinyML platform: Data sampling distribution and classifier training results with synthetic aroma data.

D Cloud Integration: Partner Cloud Infrastructure and ThingsBoard

D.1 Partner Cloud & Broker Connection (Class C Collaboration)

To realize the distributed IoT architecture:

- The ESP32-S3 securely connects to the cloud MQTT broker via TLS v1.2 encryption.
- Telemetry data is packaged as structured JSON payloads and published to the primary telemetry topic:

```
Topic:  aromalab/esp32/prediction
Payload: {"voltage": 2.14, "prediction": "Kopi Arabika", "device_id": "ESP32-S3"}
```

- The partner's cloud infrastructure (Class C / Azure backend) consumes this topic, performing persistent logging and database indexing for analytical reports.

D.2 ThingsBoard IoT Platform Integration

In parallel with the partner cloud backend, the device was successfully connected to the **ThingsBoard IoT Dashboard Platform**:

- **Telemetry Ingestion:** Real-time sensor voltage and classified aroma attributes are streamed directly into ThingsBoard using MQTT device credentials.
- **Interactive Dashboard:** A rich visual dashboard was configured with real-time digital gauges for instant voltage monitoring, time-series line charts tracking VOC progression, and status label widgets indicating the current aroma state.
- **Alarm Triggering:** High-concentration alerts were implemented to notify when gas levels cross safety thresholds.

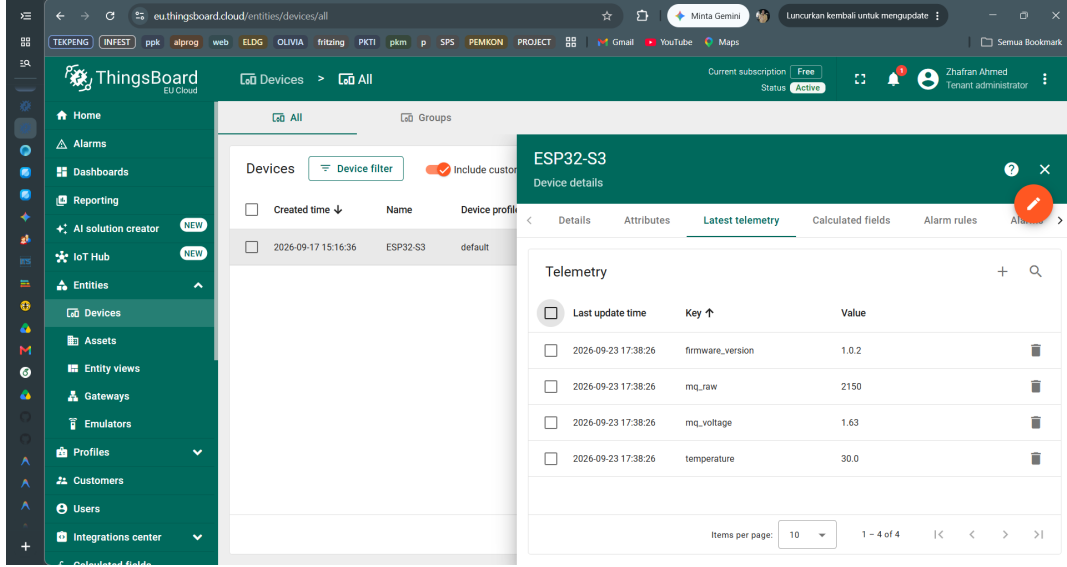


Figure 4: ThingsBoard cloud dashboard displaying real-time sensor metrics, telemetry history, and aroma classification state.

E Over-The-Air (OTA) Updates and Dynamic Firmware Switching

A hallmark milestone of this project is the implementation of fully remote, seamless Over-The-Air (OTA) updates. This capability allows the device firmware to be updated, reconfigured, or completely switched over Wi-Fi without requiring physical USB connection.

E.1 OTA Architecture and Mechanism

The OTA workflow operates as follows:

1. An update trigger script publishes a control payload to the dedicated MQTT topic:

Topic: `aromalab/esp32/ota`

Payload: `http://<HOST_IP>:8000/<firmware_image>.bin`

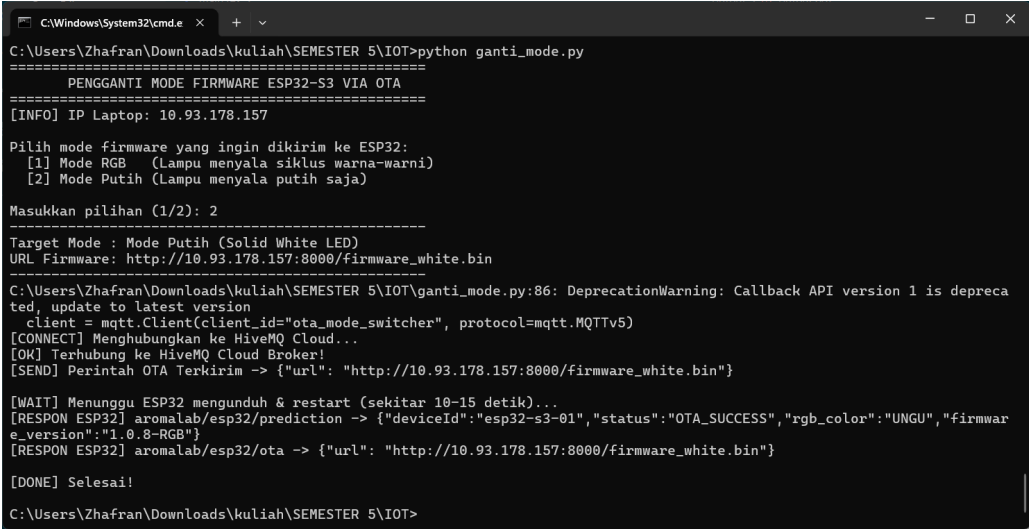
2. The ESP32-S3 receives the URL in its callback loop, initializes the ESP-IDF OTA client, and writes binary streams chunk-by-chunk to the inactive flash slot (e.g., writing to `ota_1` while running on `ota_0`).
3. Upon successful validation and SHA-256 verification, the active boot pointer in `otadata` is switched, and the microcontroller automatically reboots into the new firmware.

E.2 Multi-Firmware Mode Demonstration

To conclusively prove the flexibility of the OTA pipeline, three independent binary firmwares were built and dynamically switched back and forth:

1. **Primary E-Nose Firmware (iot.bin / v1.0.5):** Executes the full sensor acquisition loop, AI prediction, and dual-cloud telemetry streaming.
2. **RGB Cycling Mode Firmware (firmware_rgb.bin / v1.0.8-RGB):** Drives the onboard WS2812 NeoPixel through a smooth color sequence (Red → Green → Blue → Yellow → Cyan → Magenta), confirming system health and OTA listener readiness.
3. **Solid White Light Mode Firmware (firmware_white.bin / v1.0.7-WHITE):** Reconfigures the NeoPixel to high-luminance solid white for illumination or visual beaconing.

Automation scripts (`ganti_mode.py` and `kirim_ota.py`) were developed to execute interactive one-click firmware switching between RGB, White, and Sensor modes.



```

C:\Windows\System32\cmd.exe x + v
C:\Users\Zhafran\Downloads\kuliah\SEMESTER 5\IOT>python ganti_mode.py
=====
PENGANTI MODE FIRMWARE ESP32-S3 VIA OTA
=====
[INFO] IP Laptop: 10.93.178.157

Pilih mode firmware yang ingin dikirim ke ESP32:
[1] Mode RGB (Lampu menyala siklus warna-warni)
[2] Mode Putih (Lampu menyala putih saja)

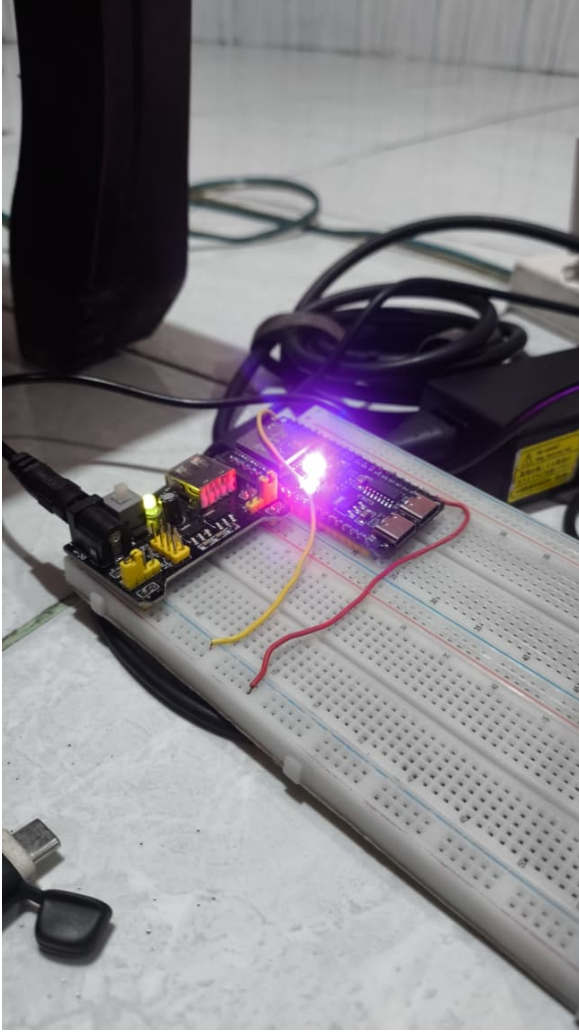
Masukkan pilihan (1/2): 2
=====
Target Mode : Mode Putih (Solid White LED)
URL Firmware: http://10.93.178.157:8000/firmware_white.bin
=====
C:\Users\Zhafran\Downloads\kuliah\SEMESTER 5\IOT\ganti_mode.py:86: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
  client = mqtt.Client(client_id="ota_mode_switcher", protocol=mqtt.MQTTv5)
[CONNECT] Menghubungkan ke HiveMQ Cloud...
[OK] Terhubung ke HiveMQ Cloud Broker!
[SEND] Perintah OTA Terkirim -> {"url": "http://10.93.178.157:8000/firmware_white.bin"}

[WAIT] Menunggu ESP32 mengunduh & restart (sekitar 10-15 detik)...
[RESPON ESP32] aromalab/esp32/prediction -> {"deviceId":"esp32-s3-01","status":"OTA_SUCCESS","rgb_color":"UNGU","firmware_version":"1.0.8-RGB"}
[RESPON ESP32] aromalab/esp32/ota -> {"url": "http://10.93.178.157:8000/firmware_white.bin"}

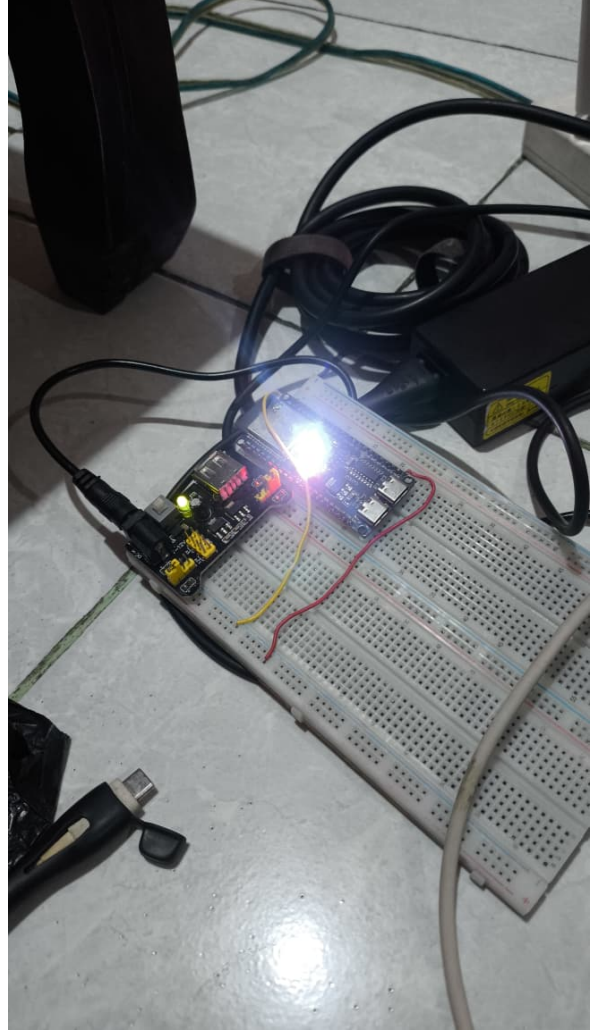
[DONE] Selesai!
C:\Users\Zhafran\Downloads\kuliah\SEMESTER 5\IOT>

```

Figure 5: Serial execution log verifying remote OTA trigger reception, binary download over HTTP, flash writing, and automatic partition reboot.



(a) RGB Rainbow Cycle Mode
(`firmware_rgb.bin`)



(b) Solid White LED Mode
(`firmware_white.bin`)

Figure 6: Physical confirmation of onboard NeoPixel state transitions accomplished exclusively via remote Over-The-Air (OTA) firmware flashing.

F Summary of Progress and Repository Documentation

F.1 Summary of Completed Milestones

Table 1 summarizes the developmental achievements realized in this project.

Subsystem	Achieved Milestone Description	Status
Embedded Firmware	ESP32-S3 bare-metal Rust firmware built with <code>esp-idf-svc</code>	Completed
Sensing	Analog gas sensor sampling on GPIO 3 ADC with filtering	Completed
Data & TinyML	Synthetic aroma dummy dataset generated and model trained	Completed
Local Actuation	Onboard WS2812 NeoPixel control on GPIO 48	Completed
Partner Cloud	MQTT TLS telemetry integration with Class C backend	Completed
ThingsBoard	Real-time dashboard streaming, chart telemetry, and alerts	Completed
Remote OTA	Full A/B dual-slot OTA updates via HTTP and MQTT triggers	Completed
Firmware Switching	Dynamic mode switching (E-Nose $\leftrightarrow$ RGB $\leftrightarrow$ White)	Completed

Table 1: Summary of completed project milestones.

F.2 GitHub Repository and Documentation

All source codes, firmware targets, Python automation tools, partition definitions, and configuration files are organized in the project repository:

- **GitHub Repository Link:**
<https://github.com/kemed99/IoT-ESP32-PROJECT>
- **Directory Structure:**
 KELAS A/ contains the complete Rust source code (`main.rs`, `main_rgb.rs`, `main_white.rs`), Cargo configuration, build tools, partition tables, and OTA client scripts. KELAS C/ is designated for partner cloud integration assets.

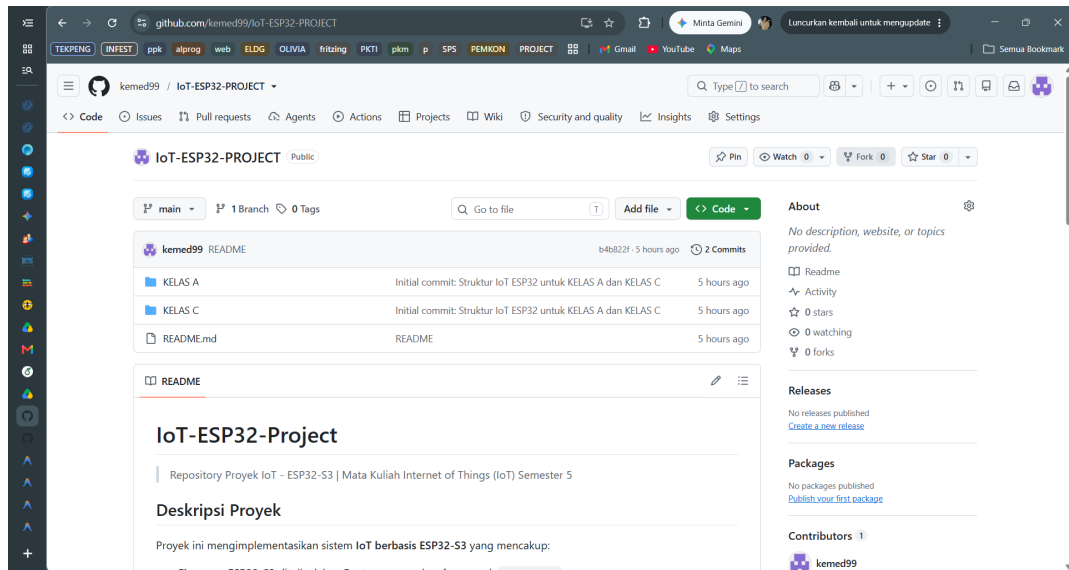


Figure 7: GitHub project repository structure (KELAS A and KELAS C) documenting open-source code and assets.

G Conclusion

This midterm progress report confirms the successful implementation of all core engineering objectives for the IoT Electronic Nose system. By leveraging embedded Rust on the ESP32-S3, high stability, memory safety, and determinism were achieved at the edge. The integration of dual-slot Over-The-Air (OTA) updates empowers frictionless remote maintenance, validated by seamlessly switching between telemetry, RGB cycling, and solid white illumination modes. The system successfully interfaces with the partner cloud infrastructure (Class C) and the ThingsBoard platform for real-time visualization, completing a robust and extensible IoT solution.